March 27, 2026

Progress Report Document

ICOM 3010 – Self-directed Study

IT Programming – Year 2

"Deserted"

Unreal Engine 5.0.3 Open World Game

A third-person, open-world, game prototype

Prepared By:

Kael Payette

Table of Contents

Project Overview 4
PROGRESS REPORT 1 5
END OF PROGRESS REPORT 1 15
Progress Report 1 Summary: 15
PROGRESS REPORT 2 16
END OF PROGRESS REPORT 2 23
Progress Report 2 Summary: 23
PROGRESS REPORT 3 24
END OF PROGRESS REPORT 3 28
Progress Report 3 Summary: 28
PROGRESS REPORT 4 29
END OF PROGRESS REPORT 4 37
Progress Report 4 Summary: 37

Table of Figures

Figure 1 – UE 5.7 Snapping Tool 6
Figure 2 – Section 1 Objectives 7
Figure 3 – Completion of Section 1 7
Figure 4 – Atmosphere, Sky & Sun Creation 8
Figure 5 – Landscape Creation 8
Figure 6 – Custom Landscape Material Editor 9
Figure 7 – Section 2 (8/17) Videos Completed 9
Figure 8 – Top-Down Landscape View (Painted) 10
Figure 9 – Black Alder Static Mesh Folder (Geometry) 11
Figure 10 – Viewport Landscape View (Added Forest) 12
Figure 11 – Top-Down View Forest in Landscape 12
Figure 12 – View of Forest from Closer to Ground (Player View) 13

Figure 13 – Foliage Editor (Asset Selection / Count View) 13
Figure 14 – Landscape Material Node Editor 14
Figure 15 – Character Asset (Player) 14
Figure 16 – Character Asset (Enemy) 14
Figure 17 – Section 2 (15/17) Videos Completed 16
Figure 18 – Preview of Valley of the Ancient 17
Figure 19 – Large Canyon Buttes 18
Figure 20 – Created Canyon Dugout for Dungeon 19
Figure 21 – Static Mesh Collisions View 20
Figure 22 – Decreased Landscape Size (Canyons No Longer Fit) 20
Figure 23 – Canyons Adjusted to New Landscape 22
Figure 24 – Coordinates 24
Figure 25 – Vector Addition 24
Figure 26 – C++ Gaming Class Inheritance 25
Figure 27 – Packages & Nesting 26
Figure 28 – Proof of Completion of Section 3 26
Figure 29 – Proof of Completion of Section 4 26
Figure 30 – Required Tools Installation 27
Figure 31 – Updating Visual Studio 2022 27
Figure 32 – Creating a Parent Class File in Unreal Engine 27
Figure 33 – Successful Visual Studio 2022 Environment Setup 29
Figure 34 – C++ Unreal Engine Code Environment 29
Figure 35 – Created Actor Items 30
Figure 36 – Begin Play Function with Custom Message 30
Figure 37 – Cleaned Up Storage 31
Figure 38 – Cleaned Up Unreal Engine Workspace 31
Figure 39 – Debug Sphere with Forward Vector 33
Figure 40 – Blueprint Editor Event Tick 33

Figure 41 – Visual Studio Event Tick 34

Figure 42 – Items.h 34

Figure 43 – Viewport View of Bird Pawn Actor 35

Figure 44 – “Bird.cpp” MoveForward Function 36

Project Overview

This document outlines the development progress of Deserted, a third-person open-world game prototype developed in Unreal Engine 5.0.3. The report covers environment creation, landscape and foliage systems, asset management, performance optimization, C++ gameplay programming, and debugging workflows completed between January and April 2026. The project evolved from environment construction into advanced C++ gameplay systems, culminating in custom pawn creation, animation control, and exposed gameplay variables through Unreal’s Blueprint system.

The objective I set out to achieve with this project was to learn about game development and produce a playable, shareable prototype as an early portfolio piece as well as a personal project to continue on my own time after I have graduated.

PROGRESS REPORT 1

Jan 7:

I started an Install of the Epic Games Launcher, once it installed, I then installed Unreal Engine version 5.7.1 as well as the Fab UE Plugin and Quixel Bridge (for bringing in assets).

- Started my first UE project the built-in tutorial to learn basics of UE
- Play mode = green play button or Alt+p (mode you test the game in)
- Edit mode = default mode, or esc once in play mode (mode you make the game in)

I learned the basics of movement:

- How to move the camera when in editing mode - up/down, forward, backward, left, right & your mouse controls looking direction
- How to interact with objects in edit mode - move (W), rotate (E), scale (R)
- How to move the character once in play mode - forward, backward, left, right - Mouse controls looking direction

---

Jan 8:

Began configuring external input hardware (PlayStation 5 controller) for use in Unreal Engine on Windows 11, this is to test out the game during and after its development. This will also make it easier for me to show the game off to people who user controllers or keyboard and mouse.

- Identified that the PS5 controller was not being recognized by Unreal Engine despite functioning on console
- Diagnosed the issue as an operating system–level input and driver compatibility problem rather than an engine or project issue
- Tested USB connections and discovered the initial cable did not support data transfer; replaced cable with a data-capable USB cable
- Successfully confirmed Windows recognition of the controller using system controller testing tools
- Installed and configured third-party input software (DS4Windows) to emulate an Xbox-compatible controller for Unreal Engine
- Identified missing virtual driver dependency (ViGEmBus) required for controller emulation and initiated installation

- Learned about differences between native controller support, XInput devices, and third-party controller emulation on Windows
- Gained experience troubleshooting hardware, drivers, and software interaction across the operating system and application layers

---

Jan 13:

Finished video 1 of section 1:

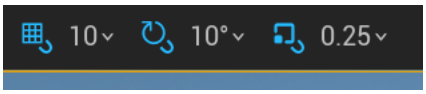
I reviewed the course goals and resources such as Q&A/FAQ and community discord, reviewed the install/setup of software and installed starter content. I had to find a work around however, as starter content was removed in UE version 5.7, however the many available resources made it very easy to find others with the same problem and their fixes. I had to install and previous version of UE, go into the content files, find the starter content and enable it; after enabling it I then copied it over to the content folder of my UE version 5.7 project.

- left mouse click = move forward, look left/right
- right click = look all directions
- both click = move up, down, left, right
- click + c = zoom in, unclick to release
- click + z = zoom out, unclick to release

Jan 19:

- Ctrl + shift + h = show fps
- 1,2,3 are location hotkeys they instantly take you to a view, 1,2,3 is default set but can be edited 0-9 by going to the desired view and pressing ctrl+(0-9)
- g hotkey = game view, how the game looks to players, f11 = Fullscreen

Figure 1: UE 5.7 Snapping Tool



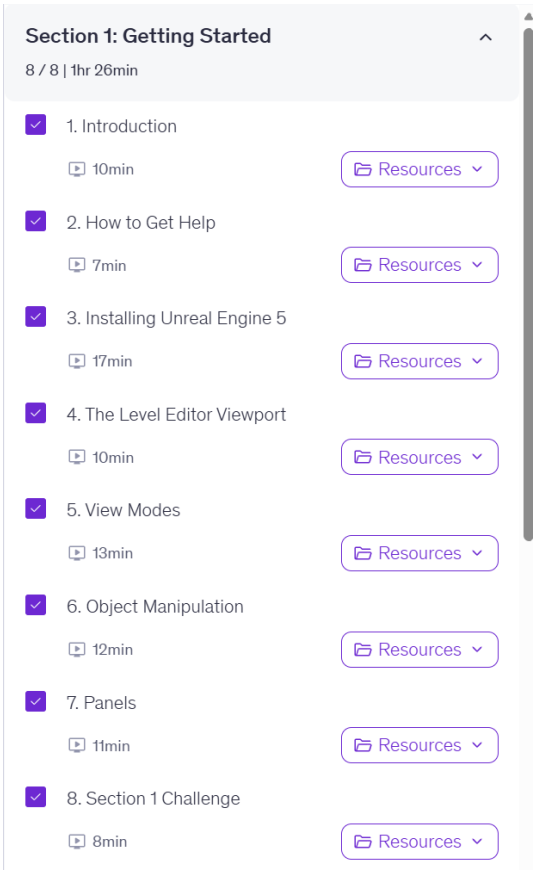
March 27, 2026

Turning off snapping is where objects move based on preset numbers as shown in *Figure 1:* when rotating, moving and scaling. This allows for precise movements/measurements when snapping is not enabled.

Figure 2: Section 1 objectives

- Shift while move object = cam moves with the object
- Shift + 1,2,3,4,5 = switches mode from select to landscape mode etc

Figure 3: Completion of Section 1



Jan 20-24:

Power outage / no Wi-Fi made it difficult to work on the project in between these times.

Due to issues arising from the landscape creation and the vast amount of differences between the version I was using (UE 5.7.1) and the version he was using (UE 5.0), I restarted the lighting/ atmosphere & landscape creation, this wasn't that bad because a lot of the first section was going over how to use the editor and the various capabilities of it. I installed UE 5.03 and restarted the landscape creation from that point, I had to all the setups initially done in section 1 (imports, and setting changes) again however, which

slowed me down, but in the end using the same or a closer version to the instructors will make the course easier to complete.

Jan 25:

I created the atmosphere and lighting for the world by using sky atmosphere(deflects light to create a realistic earth atmosphere affect to lighting), skylight() and 2 suns (this was a personal choice you can have up to 2 suns and customize their positioning and colour temp to achieve vastly different lighting for games; I went with an orange desert theme). The clouds are a built in feature of UE and don't need to be manually added and configured. The landscape is 32x32,

Figure 4: Atmosphere, Sky & Sun

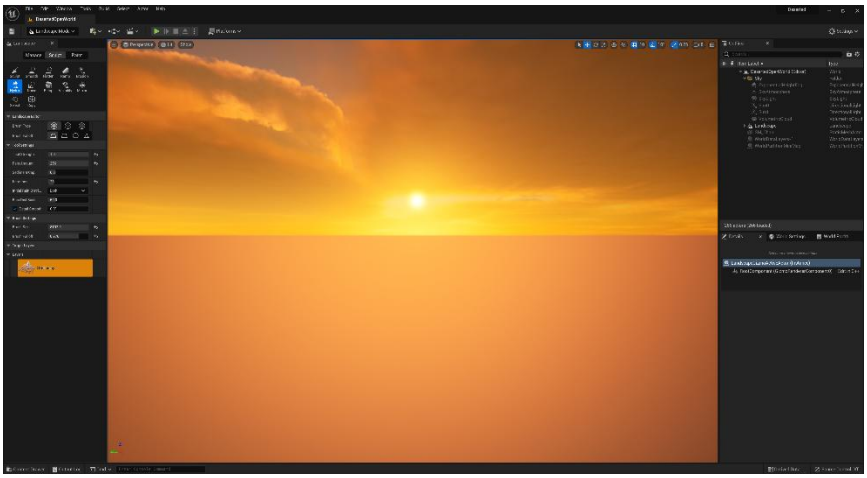
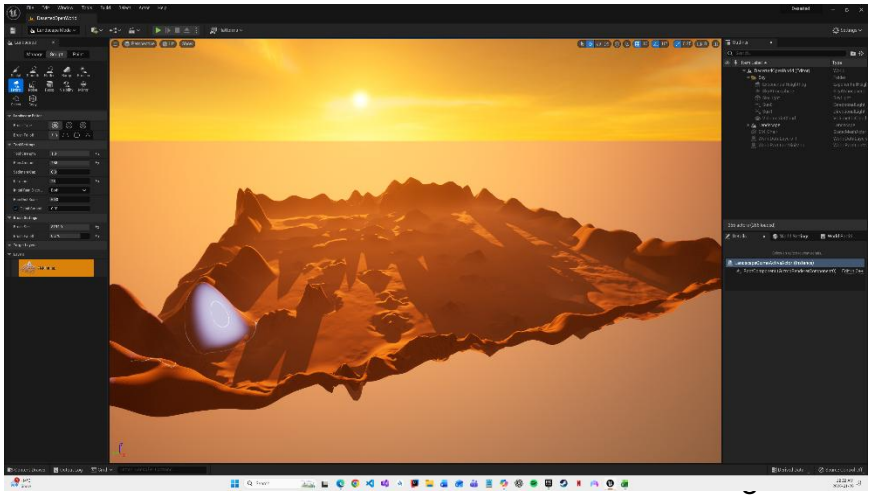


Figure 5: Landscape Creation



Jan 27:

Learned how to map textures so that they can be used to paint the landscape with. Base colour texture plus normal texture is mapped to allow them to be used in a brush, they are given custom names. Each entire texture is customizable; by double clicking on it you can edit various aspects of the texture. You can easy swap out textures for others by replacing the 2 elements with the new ones and simply deleting the previous unwanted texture.

Figure 6: Custom Landscape Material Editor

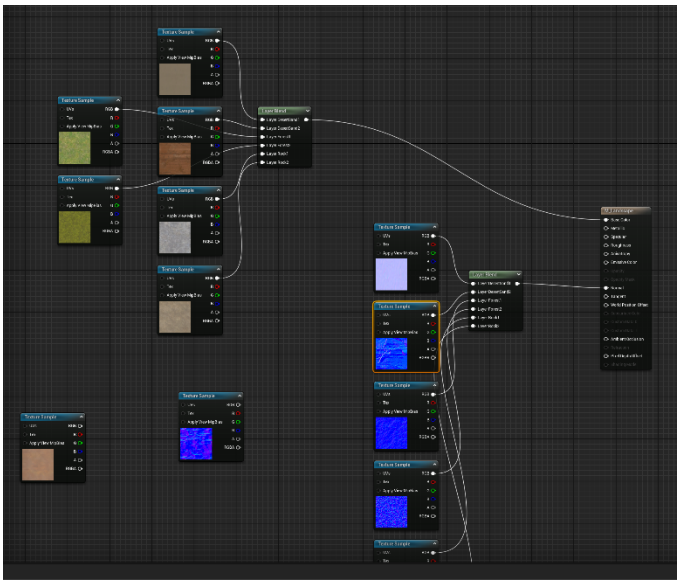
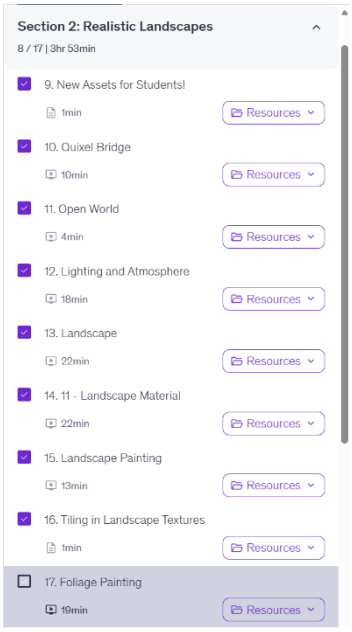


Figure 7: Section 2 (8/17) Videos Completed



I started looking at foliage editor mode as well as importing various grass textures and trees. I had to use a work around like all the other assets as the Quixel store is no longer fully supported, but the assets can be accessed using a different method, the fab store. The fab allowed me to download the assets I needed but due to the lack of information in the video regarding this method, as he uses a different one, I ran into some issues which slowed me down.

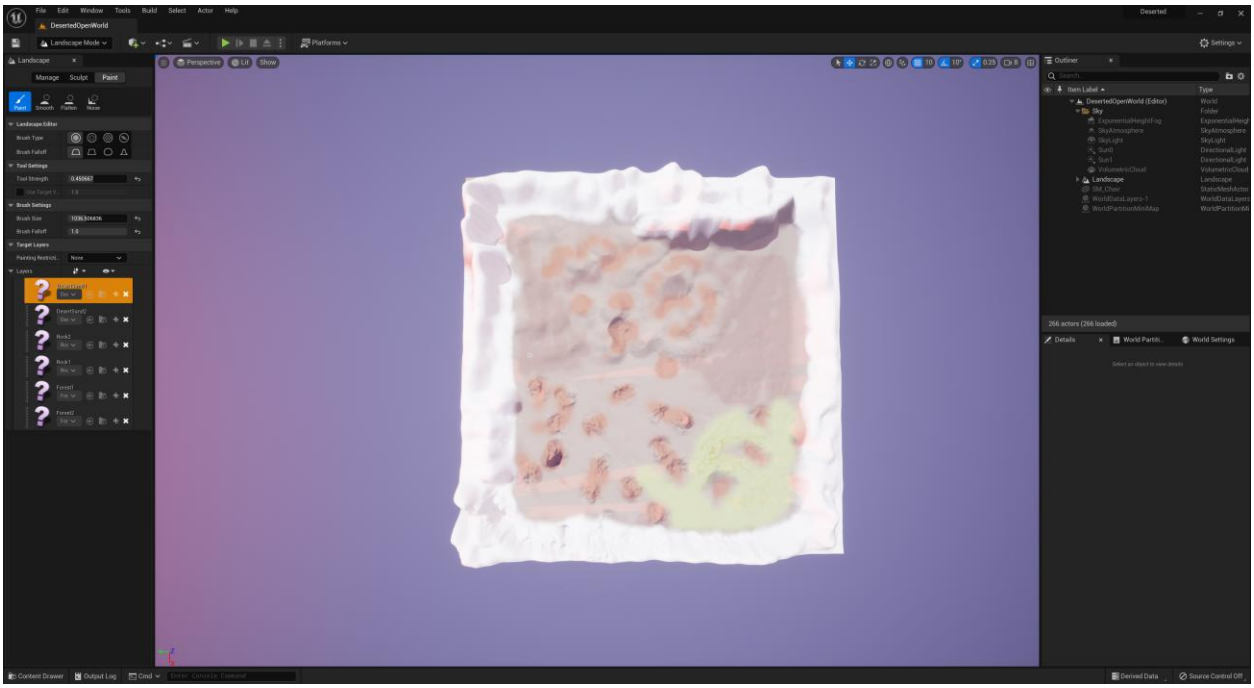
I did however create a sand, snow and rock texture for my map through painting as you see in the photo below.

Jan 29:

I worked on report 1, formatting and adding in detail. I experienced 3 crashes whilst trying to open specific levels or various things in the engine. I later resolved this issue when I discovered the crashes were linked to my trying to delete a specific asset, so I simply stopped trying to delete that asset, and that has worked for me so far.

I applied the base paint layer for the landscape with my downloaded materials, and I changed the locations of the two suns to achieve more of a purple sky, as I found the orange tone was making it hard to see a lot of the colour from the materials on the ground. The outer mountain range of the landscape was painted with a snow texture, the middle region a sand texture and there is also forested section fitted in a corner. I filled in some craters with canyon type material to further differentiate it from the rest of the landscape and add to the deserted planet vibe. I learned how brush size and density affect how a material is painted onto the landscape.

Figure 8: Top-Down Landscape View (Painted)



Jan 30:

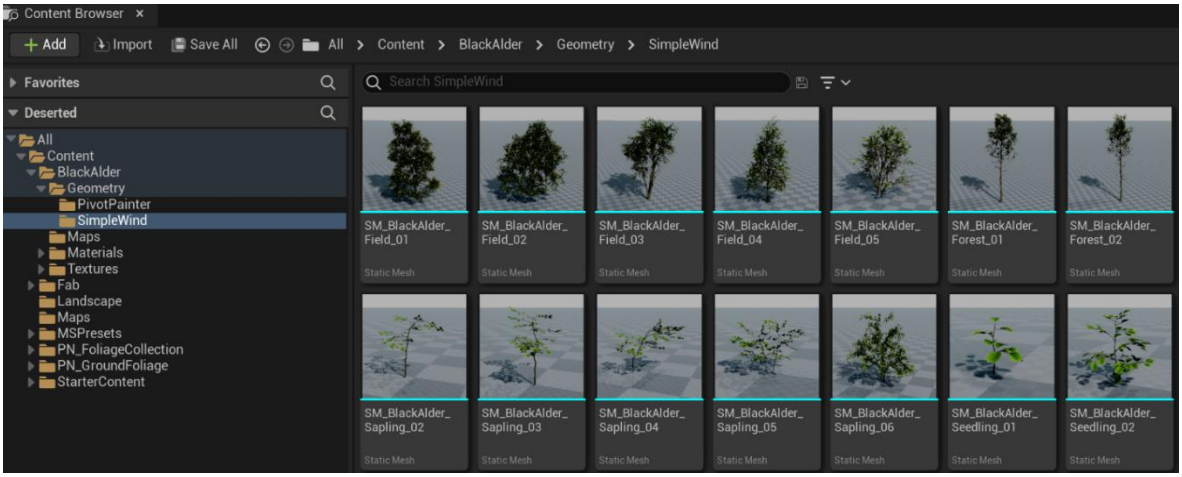
Spent the last 2-3 days attempting fix the foliage issues, the changes made to Quixel Bridge which removed it access made it hard to follow what the course instructor was doing/using. His provided assets were for UE version 5.4+ and therefore did not migrate into my 5.0 project properly:

- Assets not appearing in folders
- Wrong asset types
- Assets being blocked due to version
- Different, always changing methods to access assets
- Foliage not being allowed to be used in paint mode

Feb 1:

Learned that assets in the Fab have a separate tab dedicated to showing the version of UE they are compatible with, if compatible with UE. You can sort by assets that are only compatible with your version of UE. I learned the various folder naming conventions of assets and the different types of meshes that can be used. I.e. static meshes are used to be placed individually into the world whereas foliage meshes are meant to be painted on in mass to your world, up to thousands even millions of assets.

Figure 9: Black Alder Static Mesh Folder (Geometry)



March 27, 2026

I downloaded many asset packs for various foliage types: from grass, to flowers, to bushes, to saplings and trees. I used them to fill the landscape with vegetation all with built-in wind movement effects. I learned about how the density and brush size affect painting foliage as well as how to make sure each tree, bush, etc grows straight up and not straight from the point they are on the ground. I built up the forest and changed various editor settings to make the performance better (fps). This was needed as the method for adding foliage like this that covers an entire landscape is to paint it on. You can change the size of the area you are painting the foliage onto as well as density of the foliage and you can select as many types of foliage as you want so the population of plants being added can diverse or all the same. The denser and larger the area being painted the more resource intensive it is to run the project and fps starts to drop off quickly. It is for this reason that the project settings were edited to enhance the performance of my computer.

Figure 10: Viewport Landscape View (Added Forest)

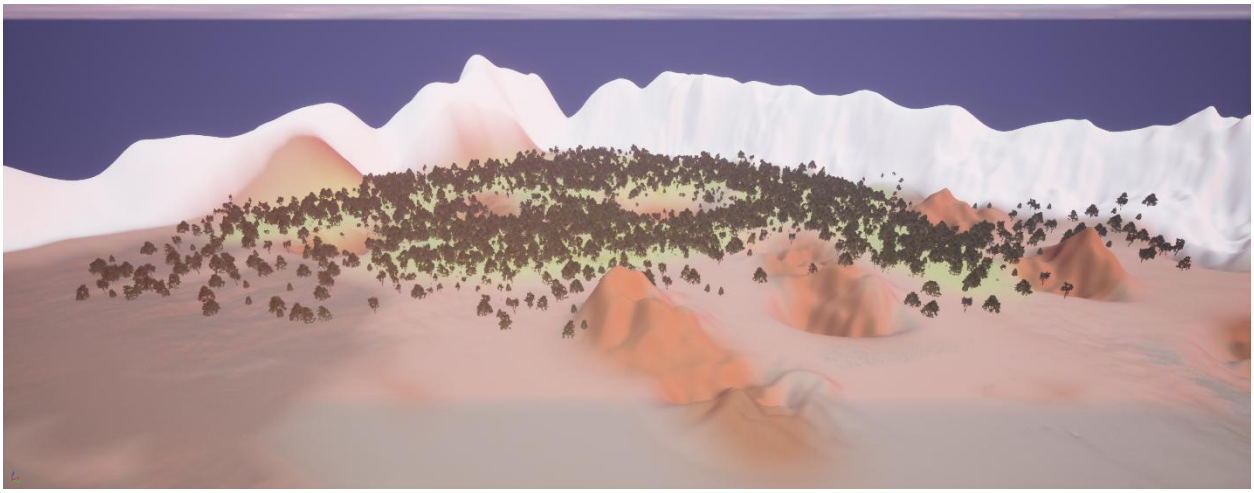


Figure 11: Top-Down View Forest in Landscape

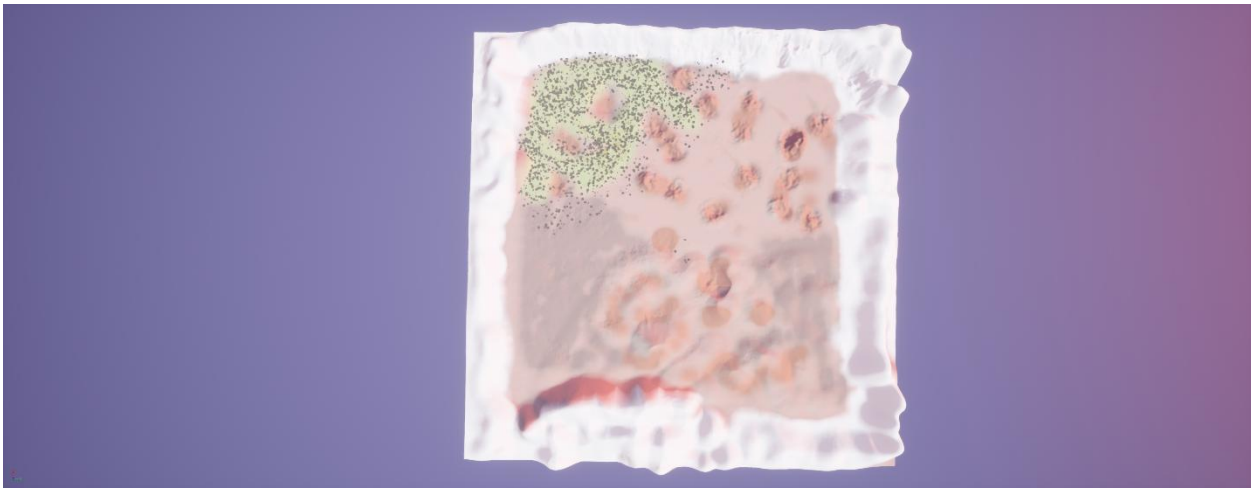
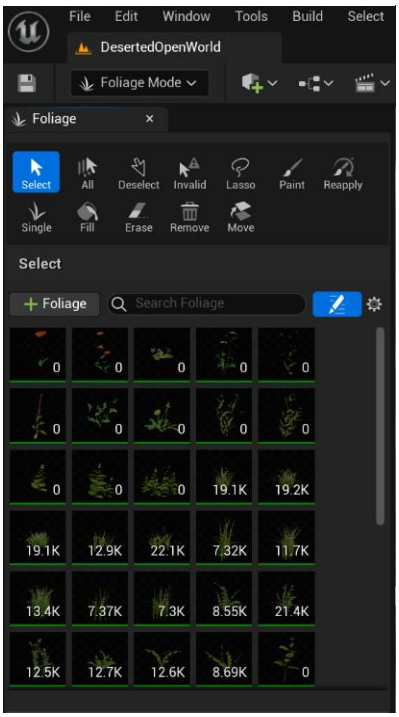


Figure 12: View of Forest from Closer to Ground (Player View)

Figure 13: Foliage Editor (asset selection/count view)



I changed various lighting settings for the project; this screenshot demonstrates the effect of the lighting changes as well as the affect being zoomed in or out has on the overall quality of the image. The suns bloom, suns positioning and colour temperature all affect how lighting affects the world.

- I also made the shadows darker to achieve a more rich and visually contrasting environment.
- F11 + g key = see and fly around in world Fullscreen.

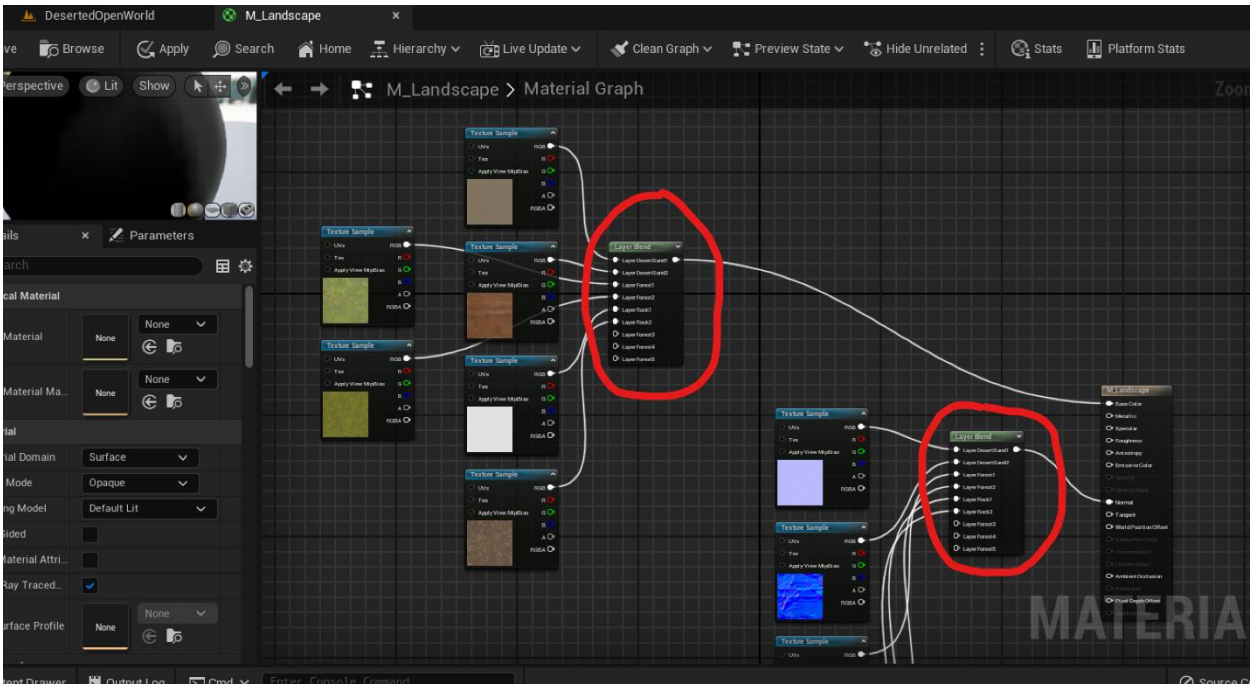
I discovered that the dynamic asset features of UE were a reason for my lowered performance whilst in the Viewport. The built-in wind affects for all foliage as well as the very advanced lighting systems made the fps drop quickly once thousands of assets were added in, you can see an example of this in this picture.

Got up to Section 2: Video 12/17.

Feb 3:

I swapped the material assets for the forest region of the map, the original was a very clean looking healthy grass, something you'd see in an empty field or lawn rather than a forest floor. I updated the "M-Landscape" which is the custom landscape material I built, which holds all the landscape materials I want to paint the landscape with, to hold more forest materials so I can swap out unwanted ones and add in new materials to give a more varied and realistic landscape. This was done by making 2 new layer blend node with the updated ability to hold 3 more materials and reconnecting all the material nodes to their corresponding asset layers.

Figure 14: Landscape Material Node Editor



I researched and found character assets and other types of assets for later use, as I want to differentiate my project from his, and achieve my more custom deserted planet theme. Below are some examples of the free to use assets available for UE 5.0 that I found:

Figure 15: Character Asset (player)

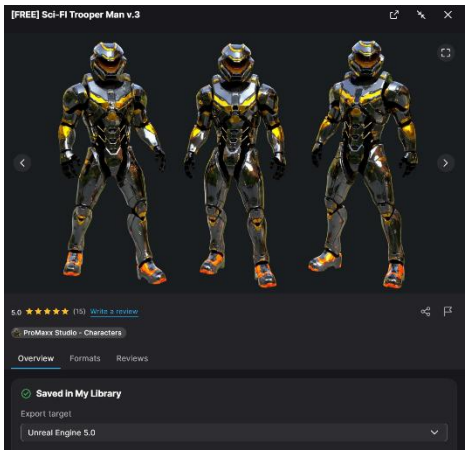
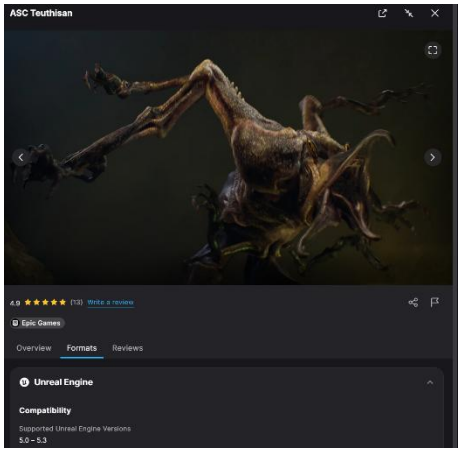


Figure 16: Character Asset (enemy)



END OF PROGRESS REPORT 1

Progress Report 1 Summary:

- Installed Unreal Engine, the Fab plugin, and Quixel Bridge, then created the first project.
- Learned core navigation: camera movement, character movement, and object controls (move, rotate, scale).
- Set up a PS5 controller for use in Unreal Engine, troubleshooting cable, drivers, and emulation software.
- Reviewed course resources and resolved the issue of missing Starter Content by using an older UE version and manually importing the assets.
- Practiced editor navigation tools such as zoom, camera look controls, and mouse-based movement.
- Learned important editor hotkeys such as FPS view, snapping, game view, full screen, and object-movement shortcuts.
- Created the initial landscape, sky, suns, atmosphere system, and learned how UE handles lighting and sky scattering.
- Built the first custom landscape material containing multiple textures and mapped them correctly for painting.
- Learned landscape painting; how brush size, strength, and layers affect ground textures such as sand, snow, and rock.
- Experimented with early foliage tools and asset import workflows, despite issues caused by changes to Quixel Bridge.
- Restarted the lighting, atmosphere, and landscape setups by switching from UE 5.7 to UE 5.0.3 to match the instructor's version, improving workflow consistency.

Notes:

C:\ProgramData\Epic\EpicGamesLauncher\VaultCache\FabLibrary - For materials

PROGRESS REPORT 2

Feb 4:

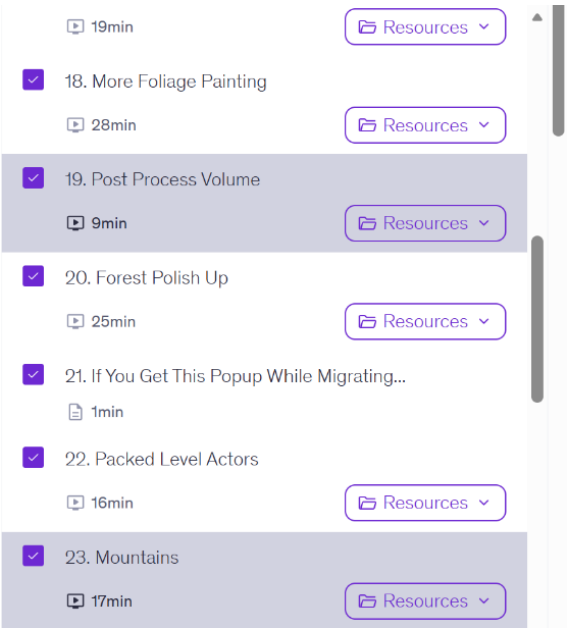
Imported and added in much more ground foliage to the forest for more detail and to hide the tiling of the materials covering the forested region of the landscape. I will watch the provided video when I find the time to fix the tiling issues with the materials. Tiling is when a texture covering a large area viewed from a distance looks very unnatural and repetitive, this is because it is essentially thousands of copies of the same small picture.

Feb 6:

Searched for and added in various bones, twigs and other objects to fill out the landscape more and create more of a deserted atmosphere.

Got up to Section 2 Video 15/17

Figure 17: Section 2 (15/17) Videos Completed



Feb 7:

Added in rocks into world, searched for and downloaded more character assets to be used as the main character as well as the various enemies around the map. I made the map much bigger, 2x larger, as with some research I found the size I was using was relatively small and that I could easily get away with making my landscape larger. It will allow for

more detail, more enemies, more stuff to explore and more for the users/players to do.

Feb 9:

Imported mountain assets, learned how to make packed level actors (many assets combined into 1 static mesh). Spent a long time searching for useable assets as everything down to the engine version and size of the world affect how the asset looks once loaded into your world. The lack of direct Quixel Bridge access also made the search harder, as the instructor was easily able to drag and drop assets as he saw fit into his world from a very large list of free assets whereas I had to spend time finding and testing each individual asset I brought in. This was wasting considerable time and not reaching the goals I wanted to achieve visually. For the time being I used the assets that were available to me, filled in the outer landscape with canyons and built up the existing landscape to hide them more.

Feb 10:

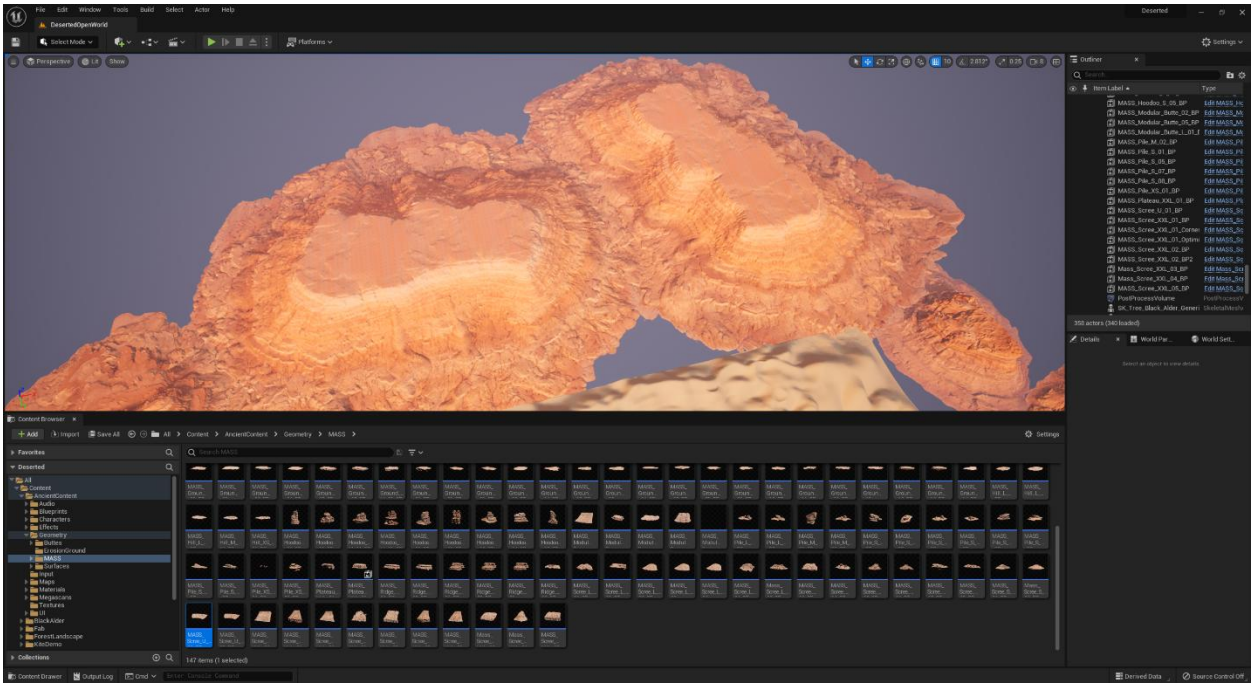
I was unhappy with the way the canyons and outer mountain range looked and luckily I discovered that a large amount of my storage space was being taken up by things I no longer needed so I removed 200+GB of VM's to clear space for the 210GB Valley of the Ancient (Valley of the Ancient | Fab) download. This allows me to have access to many of the assets that will be used in the remainder of the course for free, which look and work much better than randomly combined assets. Below is a showcase of the project:

Figure 18: Preview of "Valley of the Ancient"

Feb 11:

I downloaded the valley of the ancient project (1.5 hr) and began using its assets to fill out the outer ranges of my landscape. These large assets are known as buttes, they are not meant to be walked on or interacted with, but to fill out the outer, non accessible but still visible background of projects. I had to create a valley of the ancient project, open it (1 hr) and then migrate the folder containing the assets I wanted to use for my landscape.

Figure 19: Large Canyon Butte's

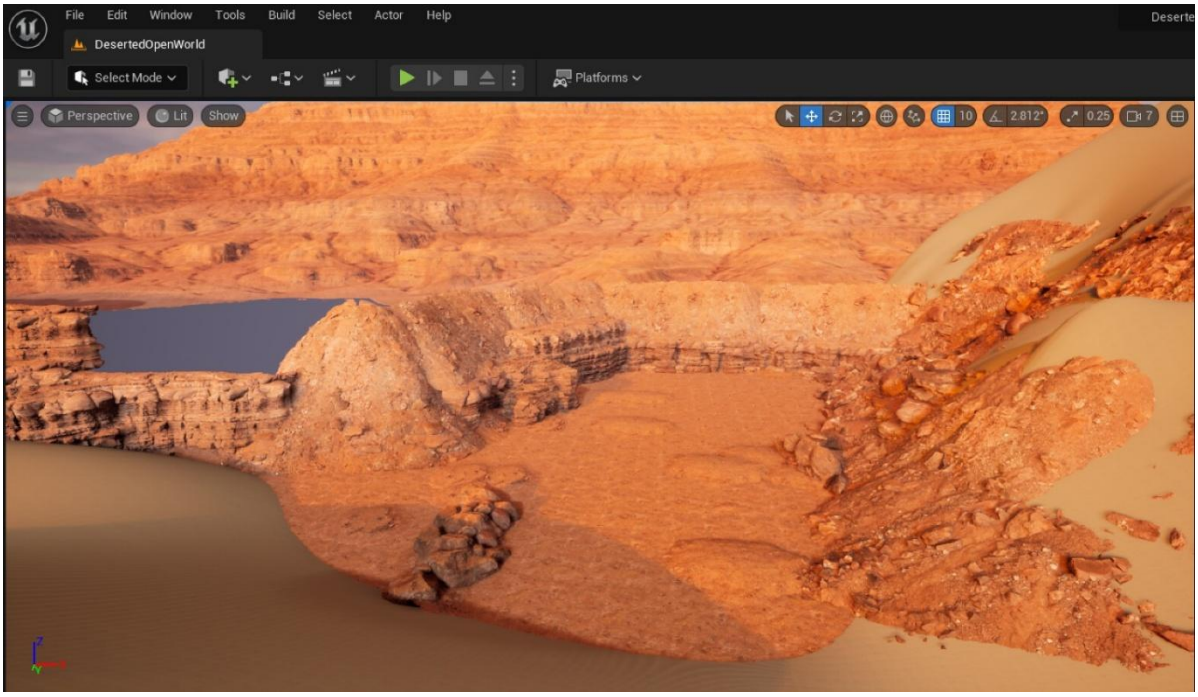


Feb 13:

Learned about collisions for the canyon ground and rocks, and how to check if objects or assets have them. I also added in the base canyon rock to serve as the platform for a separate dungeon level that will be added in using the Valley of the Ancient | Fab download in the next lesson. The portion I am going to begin now will take a significant amount of time, as I am going to cover the entire outer edge of my landscape (the mountain region) with the canyon assets brought in, this makes the entire world look much better and much more detailed. This will also take more time due to the increased size of my landscape, now 2x bigger in all directions.

This little dug out region will be the location of the dungeon, the ground and slopes all have collision so the player can walk on them and not fall through the world, whereas the unreachable outer canyons do not have collisions, this reduces the load of the game and helps it perform better. Similar to many assets having blank or solid colour backs, the back of the object is never meant to be viewed by players, this is to to save on memory space and all that.

Figure 20: Created Canyon Dugout for Dungeon



Feb 14:

Worked Feb 13, 14 and completed other due assignments on Feb 15 and 16.

Feb 17:

I began to build up the outer wall layer with canyon assets; The grey pattern you see in the image below is the view with collisions made visible. Objects with this grey pattern have collisions and players will not phase through or walk through them; they will block them like real physical objects. I shrank the landscape back to its original size as I found it was taking much longer to complete, because of the increased size, and because I brought in the assets from "Valley of the Ancient" I no longer need a scaled up world to accommodate for very large assets, as those were the only ones available.

Figure 21: Static Mesh Collisions View

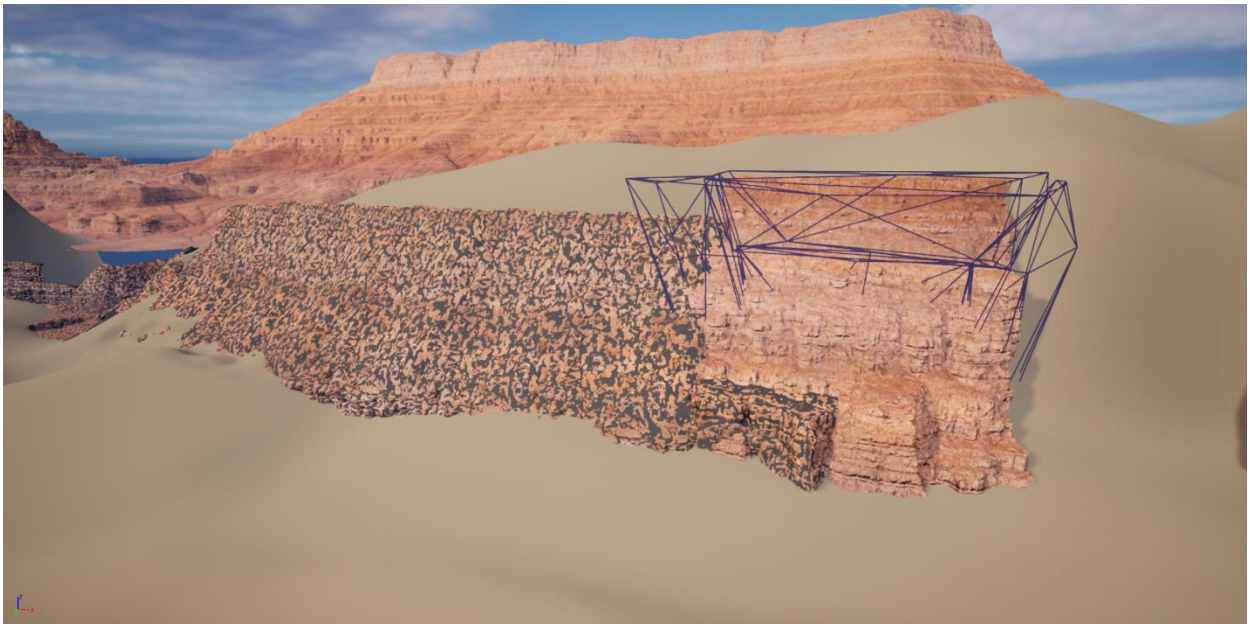


Figure 22: Decreased Landscape Size (Canyons No Longer Fit)

March 27, 2026

I had to readjust some things, and will have more to readjust because of these changes, but with my newly gained knowledge of UE it is much easier to find and fix these errors or mistakes.

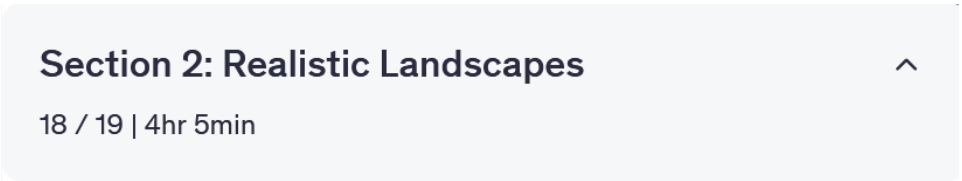
I changed the Light colour and light intensity of sun 1 to create a more other wordily sci fi kind of lighting for the world.

The next changes I will make are:

- Finish the outer edge of the landscape with the canyon walls
- Add in and paint the ground near the canyon walls with the canyon material to have a more realistic seamless landscape look
- Adjust the landscape to better fit the new size, aka forest is right near dungeon and trees are currently growing from canyon rock.

Feb 18:

Section 2: Video 18/19



I discovered the instructor of the course has recently added in 2 additional videos into this section which covered more in-depth foliage editing as well as help with importing assets and the changes to UE 5.7 (no longer applicable to me as I have switched back to 5.0.3)

I removed a lot of trees and foliage to make the forest less resource intensive and to trim the forest to fit the new landscape size better. I also adjusted the entire outside edge of the landscape to better fit the canyon rock walls being added.

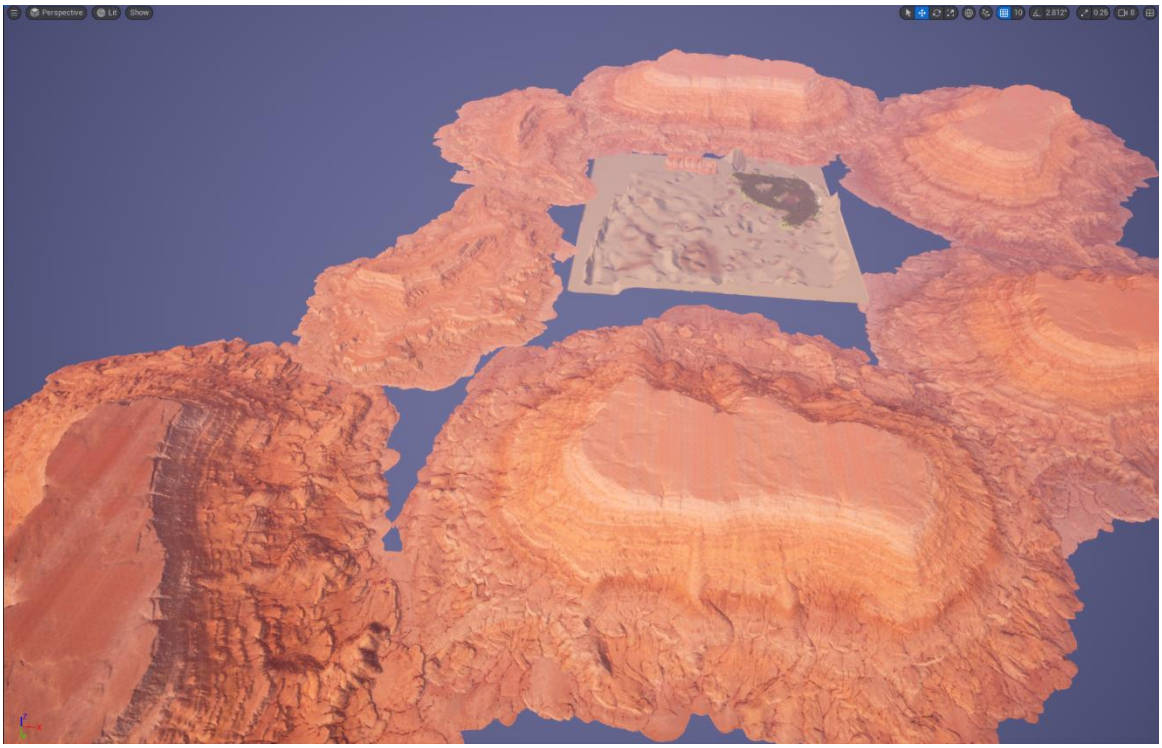
March 27, 2026

I imported a free dungeon level from the Fab to use in my world, the dungeon is complete with inside lighting, details, assets, etc.

Dealt with multiple engine crashes while attempting to edit a duplicate of the Dungeon level. The dungeon level has to be added as it has many features that need to be removed or changed before adding it into my world, such as:

- Outside lighting
- Light blocking assets
- Camera assets for cinematics

Figure 23: Canyons Adjusted to New Landscape



END OF PROGRESS REPORT 2

Progress Report 2 Summary:

- Added more ground foliage to improve forest detail and reduce visible texture tiling.
- Added bones, twigs, and other small environmental props to strengthen the deserted-planet atmosphere.
- Continued through the course, reaching near the end of Section 2.
- Imported new rock assets and downloaded more character assets for both the player and enemies.
- Expanded the map to twice its original size after researching ideal world sizes.
- Imported mountain assets and learned how to combine many meshes into one using packed level actors.
- Spent time troubleshooting asset compatibility issues caused by engine version differences.
- Freed over 200GB of storage by removing old VMs to download the Valley of the Ancient (210GB) for high-quality assets.
- Migrated Valley of the Ancient assets into the project and used them to rebuild the outer mountain ranges with better results.
- Learned and tested collision settings for canyon assets and built the base for the dungeon entrance area.
- Filled in more canyon structures to outline the world's borders.
- Reduced the landscape size back to normal to improve workflow and asset placement.
- Adjusted lighting settings, including sun color and intensity, to create a more sci-fi atmosphere.
- Cleaned up forest density and foliage placement to suit the resized landscape and improve performance.
- Imported a complete dungeon level from Fab, with lighting and details already built in.
- Encountered multiple engine crashes while removing unneeded dungeon lighting, camera rigs, and cinematic assets.
- Continued reshaping terrain around the canyon walls and forest edges to better match the new world layout.
- Began finalizing the outer canyon ring and repainting ground material around it for a seamless look.

PROGRESS REPORT 3

Feb 22 & 23:

Worked on completing the final version of presentation one and completed practice runs to prepare for the real in class presentation.

Feb 24:

I worked on cleaning up my landscape as well as the dungeon section and I attempted to fix the bug/s that causes UE to crash when I open the dungeon map. I have not resolved this issue yet, and it has slowed me down. In the meantime, I added more canyon walls around the map, tightened up the forest and removed foliage assets that were not in the forest, which occurred from me resizing the world twice.

Feb 25:

I again attempted to fix the dungeon bugs to no success, so I decided to skip that portion to save time and allow myself to move on. I can also always go back an add the dungeon later if I want to as it is quite simple to edit and add in the additional level once the bug is resolved. I finished up the landscape section to begin moving onto the other sections of the course.

March 6:

- Coordinates: (x, y, z) **|** origin = (0, 0, 0)
- Scalar = a single numerical value
- Vectors – direction and magnitude
- Vector addition and subtraction in 2D and 3D
- Vector multiplication and division in 2D and 3D
- Vector plus Vector
- Vector subtract Vector
- Vector operations 2D & 3D
- Pythagorean theorem:
- A2+B2=C2
- C=sqrt(B2 + A2)

Figure 24: Coordinates

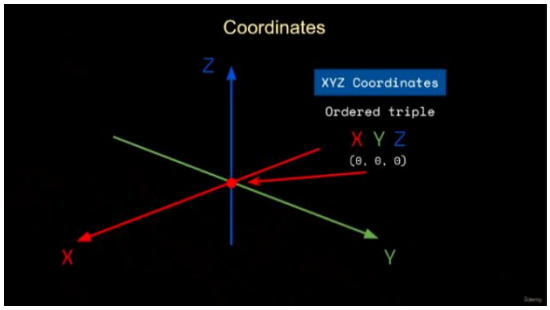
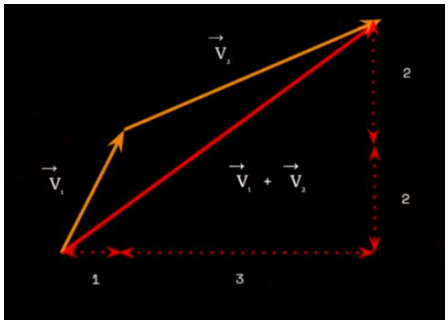


Figure 25: Vector Addition



March 7:

Vector magnitude 3D – important for calculating the distance between things and calculations for various game logic, for example. In a fishing game, the distance a fish swims away from you correlates with if the rod has been pulled into the water or not. An accurate calculation is needed to find that distance in a 3d plane.

- Vector = sqrt(x2 + y2 + z2)
- Unit vector = vector with a magnitude of 1
- Normalization of vectors
- Left vs right hand coordinate system
- UE uses left hand

Rotator: stores these in degrees

- Pitch = rotation about the Y axis
- Yaw = rotation about the Z axis
- Roll = rotation about the X axis

I choose the IDE I will be using to develop the projects C++ code with; I'm going to choose the IDE the instructor chose, which is Visual Studio. I have already installed it onto my pc as I have used it for the C# class. I may switch over to Visual Studio Code later, if I feel the need too.

Resolved versioning issues with Visual Studio Community Edition (the free version). A download for the right version for UE 5.0 (VS 2022) wasn't available from the official Microsoft page, so I found a download here:

- Reddit
- https://www.junian.net/dev/visual-studio-community-download-links/
- Configured various Visual Studio settings for UE C++ game development
- 15-minute refresher of C++ inheritance, polymorphism and relationships.
- Parent class – Child class

Figure 26: C++ Gaming Class Inheritance

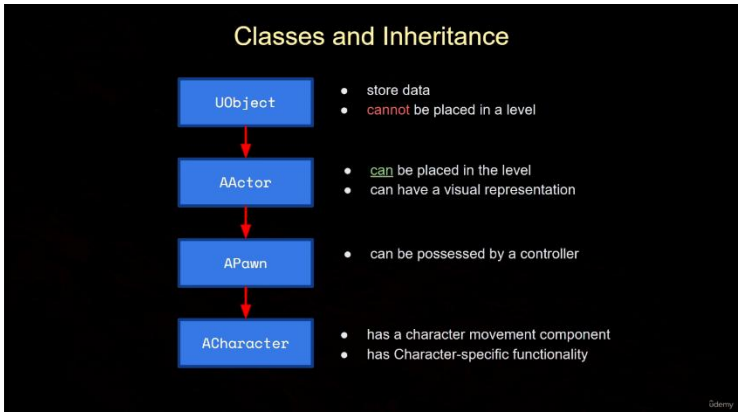
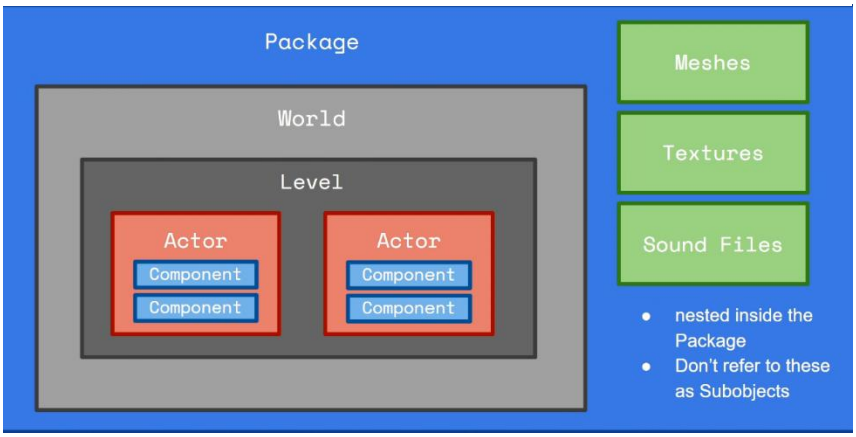


Figure 27: Packages & Nesting



- Reflection: Harvesting data for UE
- Garbage collection
- Macros: UCLASS = allows for participation in garbage collection.

Figure 27: Proof of completion of Section 3

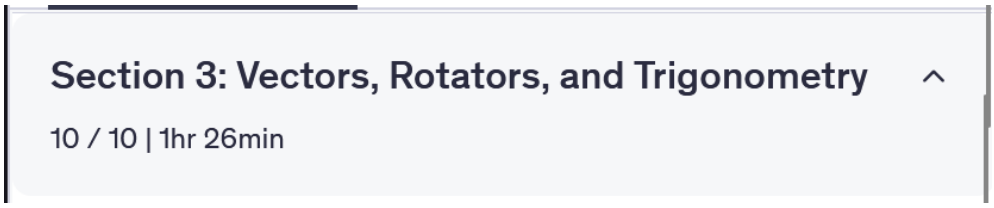
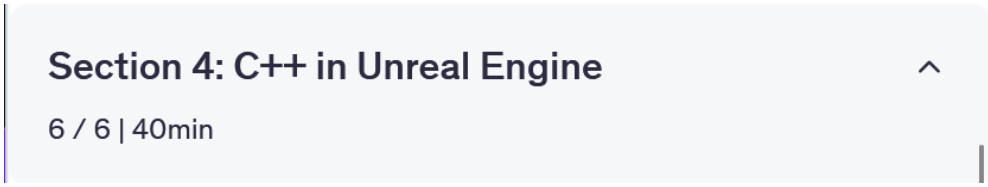


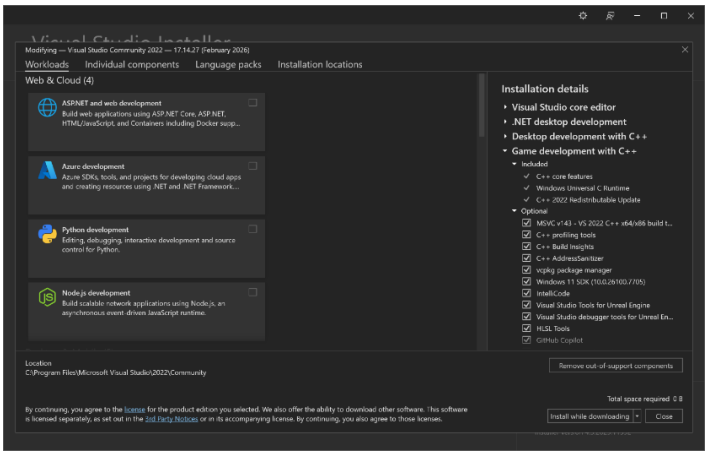
Figure 28: Proof of completion of Section 4



March 10:

Worked with Copilot AI to resolve issue preventing me from linking my UE project with visual studio 2022, I was missing an SDK and a compiler tool which are crucial for this step. Took around 1.5 hours.

Figure 29: Required tools installation



March 27, 2026

Figure 30: Updating Visual Studio 2022

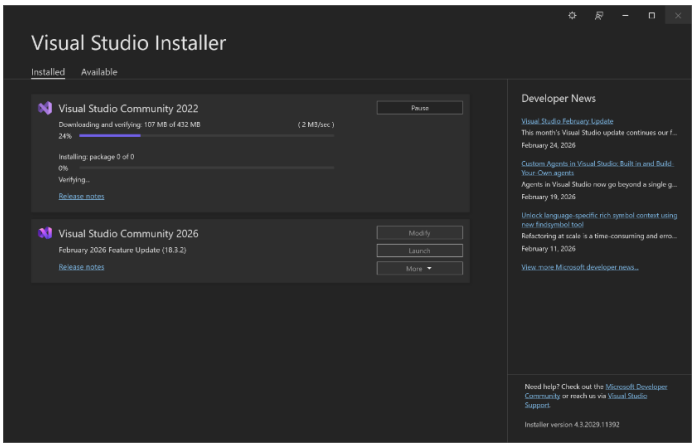
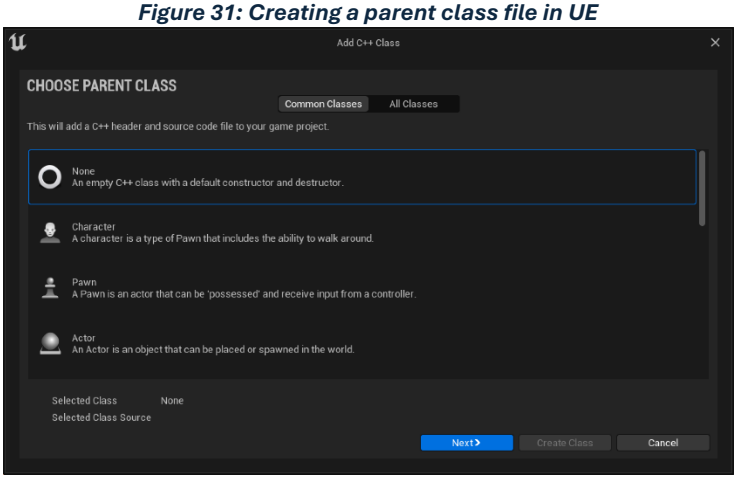


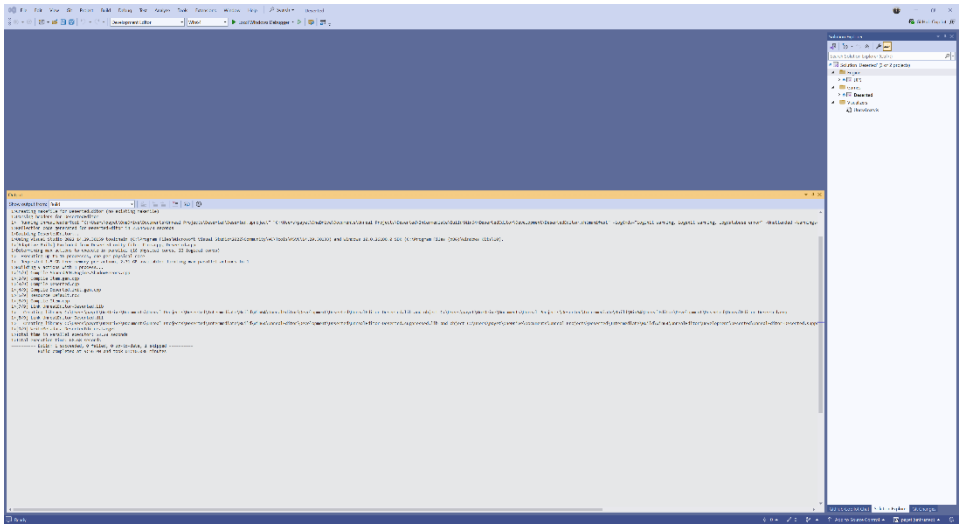
Figure 31: Creating a parent class file in UE



March 12:

I restored a broken Unreal Engine 5.0.3 C++ environment by diagnosing a compiler mismatch, installing the correct MSVC v143 toolset, cleaning the project, regenerating solution files, and successfully rebuilding the project in Visual Studio. This resolved the rebuild loop and allowed the project to open normally again. This should be the last environment setup step, which will allow me to begin programming with C++.

Figure 32: Successful VS 2022 environment setup



END OF PROGRESS REPORT 3

Progress Report 3 Summary:

- Completed the full landscape section, including repainting, reshaping terrain, and rebuilding the outer canyon ring using Valley of the Ancient assets.
- Improved world atmosphere through updated lighting settings, including sun color, intensity, and shadow adjustments to match the sci-fi theme.
- Optimized foliage density and placement to improve performance and better fit the resized landscape.
- Resolved multiple Unreal Engine crashes by identifying asset conflicts, version mismatches, and corrupted dungeon level files.
- Migrated high-quality assets from Valley of the Ancient to replace earlier placeholder assets and improve overall world quality.
- Learned advanced foliage workflows, collision systems, asset compatibility rules, and performance-related editor settings.
- Reviewed and practiced vector math concepts (2D/3D magnitude, normalization, coordinate systems, rotators) in preparation for upcoming gameplay programming.
- Set up and repaired the full C++ development environment, including fixing MSVC toolset issues, regenerating solution files, and successfully linking Unreal Engine with Visual Studio 2022.
- Imported a complete dungeon level and began modifying it, despite persistent crashes that required temporarily skipping the dungeon section.
- Continued refining the world layout by adjusting forest boundaries, canyon walls, and terrain transitions for a more cohesive environment.

PROGRESS REPORT 4

March 18:

Began developing the C++ code portion of the UE project. This code is automatically created when you convert a non-C++ UE project to a UE project that uses C++.

Figure 33: C++ UE Code Environment

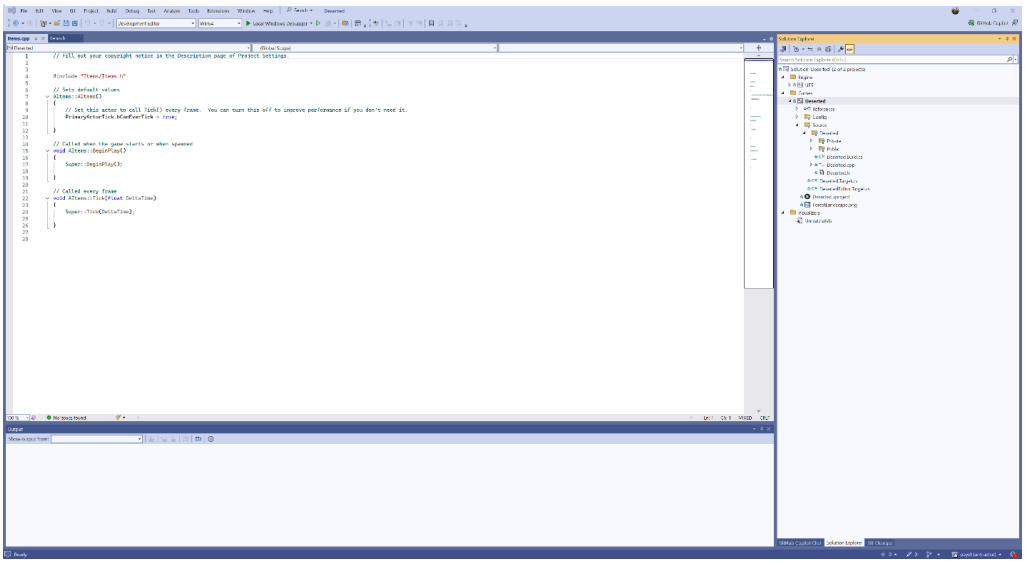
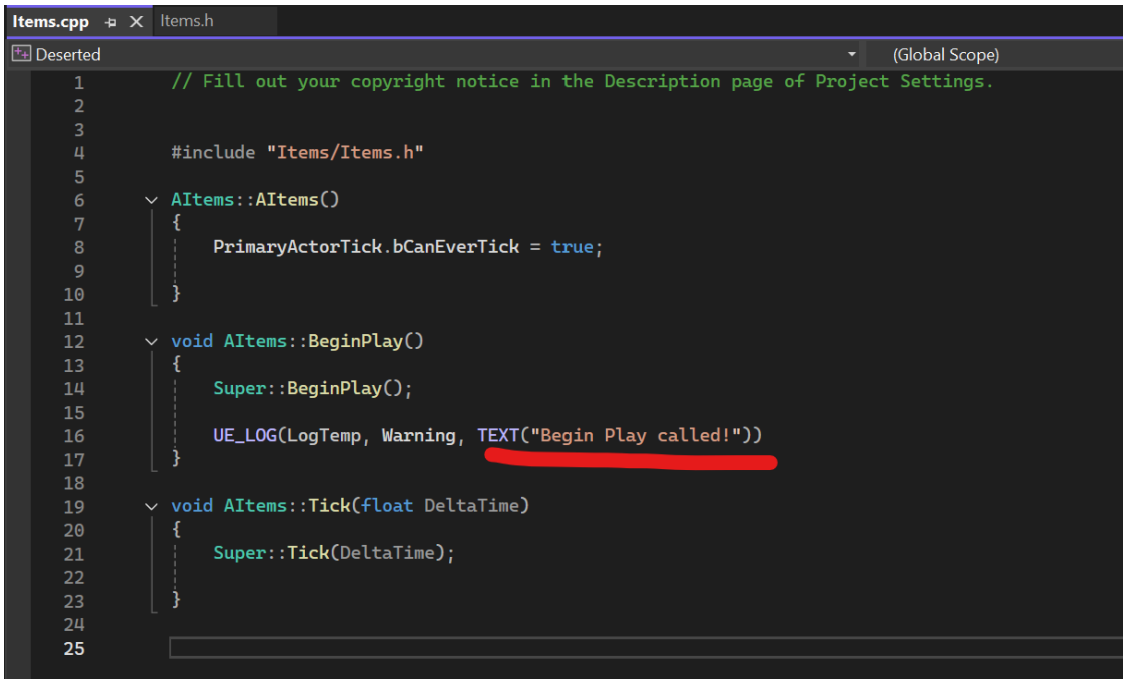
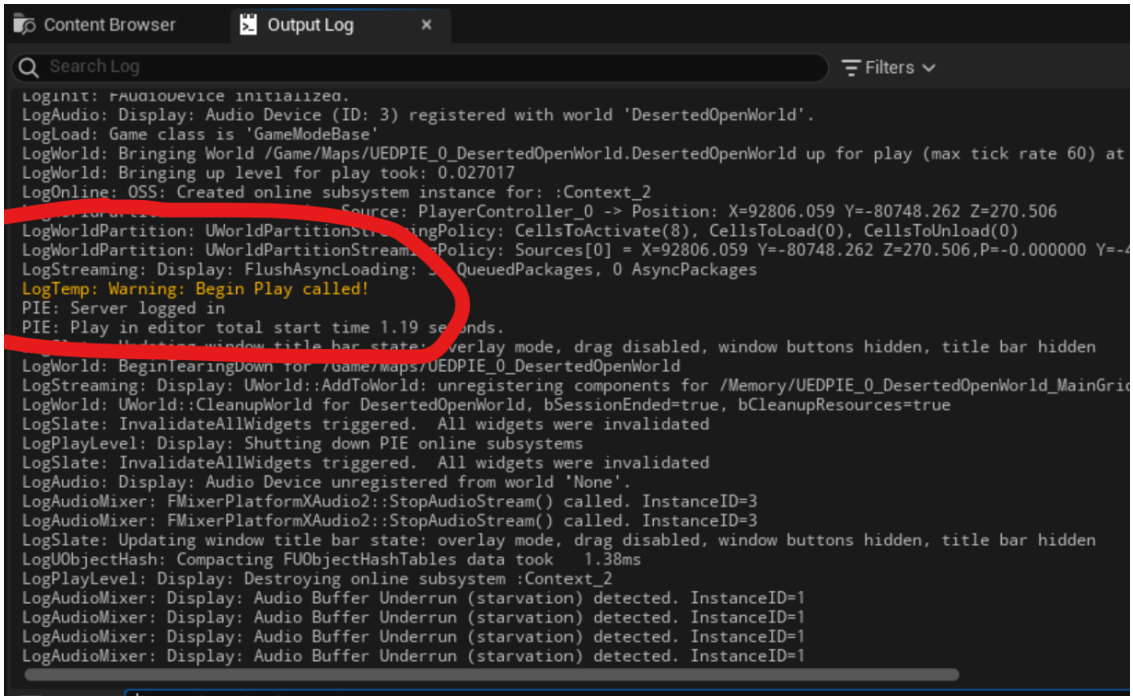


Figure 34: Created Actor Items



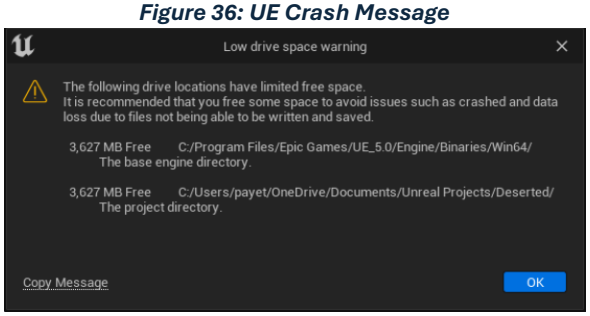
```
// Fill out your copyright notice in the Description page of Project Settings.


#include "Items/Items.h"

AItems::AItems()
{
    PrimaryActorTick.bCanEverTick = true;

}

void AItems::BeginPlay()
{
    Super::BeginPlay();

    UE_LOG(LogTemp, Warning, TEXT("Begin Play called!"))
}

void AItems::Tick(float DeltaTime)
{
    Super::Tick(DeltaTime);

}
```

Figure 35: Begin Play Function with custom message



I ran into a video card memory issue which caused the editor to crash 3 times. This error was caused from once again having to little storage on my computer as well as the amount of assets being used in my world.

Figure 36: UE Crash Message



March 19:

World Partition & Streaming

- I re-enabled World Partition streaming to fix missing assets and improve world loading. This only loads in specific parts of the world you select, rather than the entire world at once, which improves performance significantly.
- Adjusted cell size from 12,800 → 6,400 for better streaming performance.
- Regenerated World Partition data and verified the grid was functioning correctly.

HLOD & Optimization

- I enabled and configured HLOD to reduce draw calls and improve distant rendering.
- I reduced shader load significantly (from ~10k to ~3k) through proper streaming and asset management.
- I improved overall editor performance by reducing the number of loaded actors at once.

Project Cleanup & Storage Recovery

- I identified two massive Valley of the Ancient sample projects (~182 GB) as unnecessary as well as 2 additional installations of different UE versions I was no longer using.
- I confirmed all required assets were already migrated into the active project.
- I moved another ~300gb of my Steam games to a different drive to free up even more space for my project.
- I freed around 415 GB of disk space in the end.
- Reviewed and confirmed other small test projects (~10 GB total) could also be safely removed to create a cleaner and clearer workspace:

Figure 37: Cleaned Up Storage

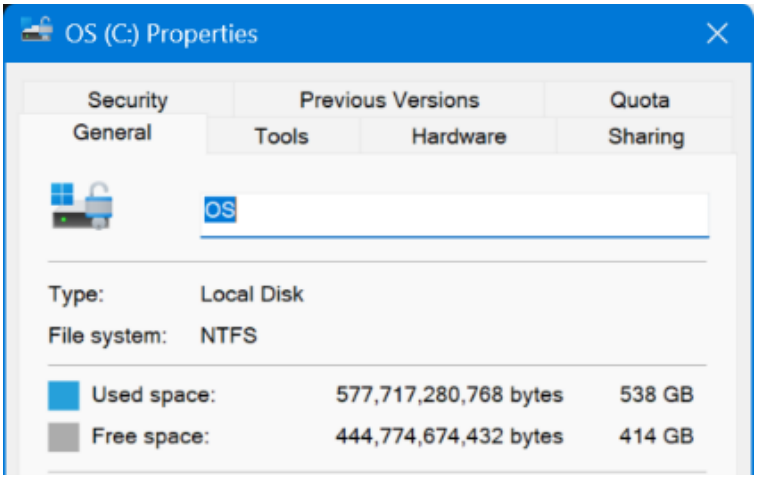
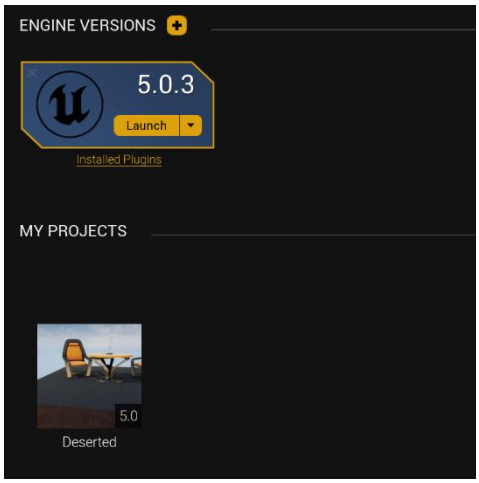


Figure 38: Cleaned Up UE Workspace



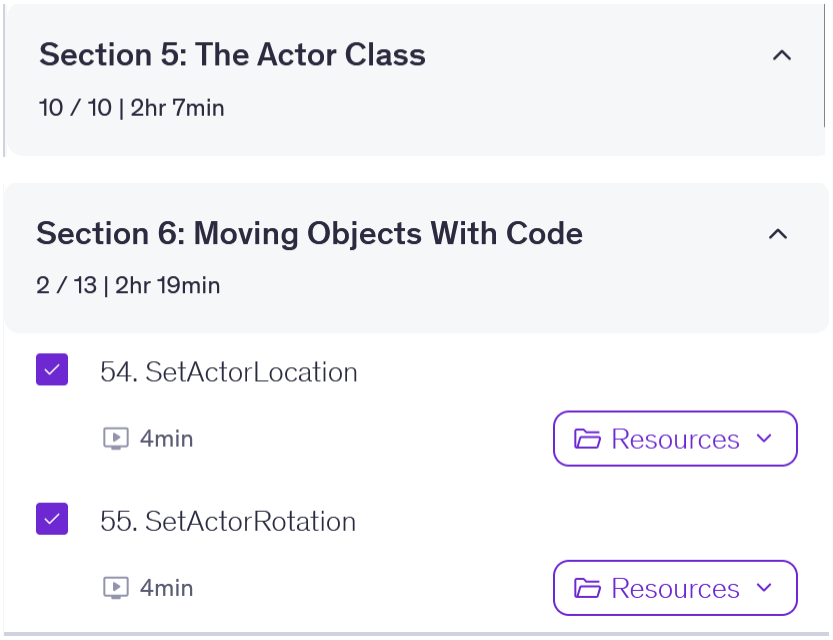
C++ Workflow Improvements

- Adopted safer compile workflow: close UE before compiling C++ changes.
- Avoided Live Coding for header changes to prevent blueprint corruption.
- Established best practice of creating new C++ classes in Visual Studio instead of the UE editor.

General Stability & Performance

- Improved world streaming behavior and reduced memory usage.
- Increased project stability by removing unnecessary dependencies.
- Ensured the project now runs on a drive with significantly more free space, improving UE performance.

Completed Section 5 & 2 videos from Section 6: Section 5 was about showing how to create, customize and use debugging tools using both UEs built in tools as well as with C++ code. A quick demo of this is provided as a video below:



UE5-Debug-Demo.mp4

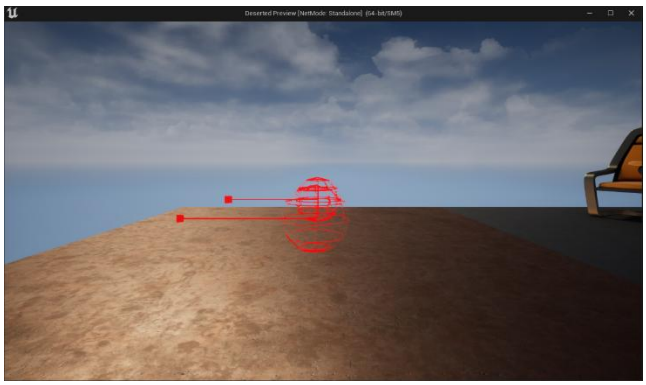
March 20:

Safest way to compile C++ code changes into the UE project:

Make C++ code changes in VS → Save & Close UE → Save & Build Solution in VS → Load Up UE. Pressing (Ctrl + F5) while in VS will save, compile then launch the UE project on a successful compile. The instructor uses a different approach for C++ code compilation (Hot share / live coding) which has since changed and he has updated advising against using and provided this simple work around instead.

The videos I completed went through how to move the debug spheres, how to add vectors to them and how to rotate them. Once again this can be achieved through the UE Event Graph or can be done in VS with C++ code, he covers how to do both but only keeps the C++ code in the end. The video below is a demonstration of the C++ code moving the debug object.

Figure 39: Debug Sphere with Forward Vector



UE5-Object-Movement-Demo.mp4

Completed 5 more videos in Section 6:

Section 6: Moving Objects With Code

7 / 13 | 2hr 19min

March 22:

The videos I covered today went over defining custom controls with C++ code and how to link them into the blueprint editor to use them in UE. This provides a lot of customizability as you can fully define every attribute of the variables. These can be things like amplitude, running time, etc.

Figure 40: Blueprint Editor Event Tick

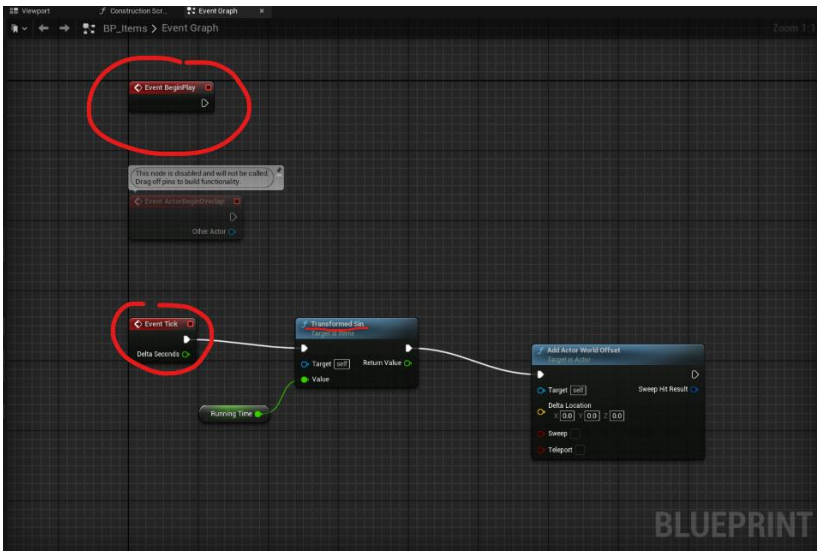


Figure 41: Visual Studio Event Tick

```
Items.cpp   Items.h
Deserted                                                    AItems
1   // Fill out your copyright notice in the Description page of Project Settings.
2
3
4   #include "Items/Items.h"
5   #include "Deserted/DebugMacros.h"
6
7   AItems::AItems()
8   {
9       PrimaryActorTick.bCanEverTick = true;
10  }
11
12  void AItems::BeginPlay()
13  {
14      Super::BeginPlay();
15
16
17  }
18
19  float AItems::TransformedSin(float Value)
20  {
21      return Amplitide * FMath::Sin(Value * TimeConstant);
22  }
23
24  void AItems::Tick(float DeltaTime)
25  {
26      Super::Tick(DeltaTime);
27
28      RunningTime += DeltaTime;
29
30
31      //float DeltaZ = Amplitide * FMath::Sin(RunningTime * TimeConstant);
32      //AddActorWorldOffset(FVector(0.f, 0.f, DeltaZ));
33
34      DRAW_SPHERE_SingleFrame(GetActorLocation());
35      DRAW_VECTOR_SingleFrame(GetActorLocation(), GetActorLocation() + GetActorForwardVector() * 100.f);
36  }
37
38
39
```

The below code outlines the permissions for the properties/variables that have been exposed to UE, they are marked with the purple “UPROPERTY” or “UFUNCTION” tags.

Figure 42: Items.h

```
Deserted                                                    (Global Scope)
1   // Fill out your copyright notice in the Description page of Project Settings.
2
3   #pragma once
4
5   #include "CoreMinimal.h"
6   #include "GameFramework/Actor.h"
7   #include "Items.generated.h"
8
9   UCLASS()
10  class DESERTED_API AItems : public AActor
11  {
12      GENERATED_BODY()
13
14  public:
15      AItems();
16      virtual void Tick(float DeltaTime) override;
17  protected:
18      virtual void BeginPlay() override;
19
20      UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Sine Parameters")
21      float Amplitide = 0.25f;
22
23      UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Sine Parameters")
24      float TimeConstant = 5.f;
25
26      UFUNCTION(BlueprintCallable)
27      float TransformedSin(float Value);
28
29  private:
30      UPROPERTY(VisibleAnywhere, BlueprintReadOnly, meta = (AllowPrivateAccess = "true"))
31      float RunningTime;
32
33  };
34
```

The videos completed today focused on defining custom controls and gameplay variables using C++ and exposing them to Unreal Engine’s Blueprint system. I learned how to use UPROPERTY and UFUNCTION macros to safely expose C++ variables and functions to the editor, allowing them to be modified and used directly in Blueprints.

This workflow provides significantly more control and flexibility, as gameplay-relevant values such as movement amplitude, timing values, and runtime parameters can be adjusted in the editor without recompiling code. This is especially useful for rapid iteration and fine-tuning gameplay behavior.

The code shown in figure 42 demonstrates how access permissions are defined for exposed properties and functions, which appear in the editor with the purple UPROPERTY and UFUNCTION indicators.

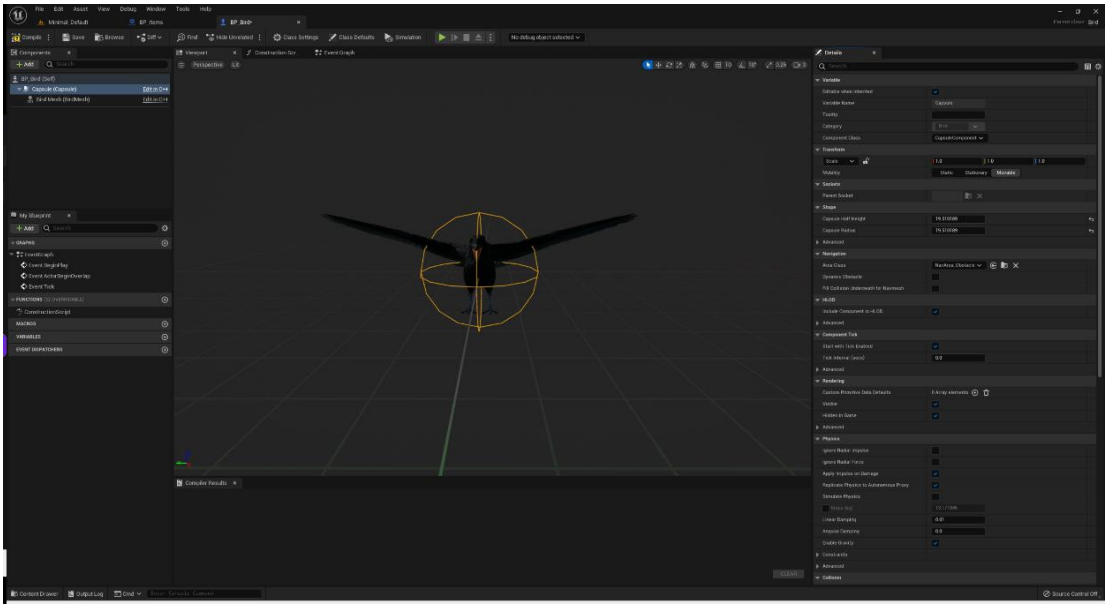
At this point in the course, I have completed Sections 1-6 (a total of 65 instructional videos) The most recent sections have been a transition from environment-focused work into more advanced gameplay programming with C++.

March 23:

I downloaded an asset pack from the marketplace called Animal Variety Pack by “”. This pack contained a bird asset with the configurations to allow it to be animated in place, it also came with a lot of animations. I created a new pawn actor in the UE which is the actor that can be possessed and controlled. I began to program in the C++ code to allow the actor to have animations and I added in the skeletal mesh for the actor, which was the bird. The reason a skeletal mesh was used and not the static mesh was because static meshes are static, aka do not move. The animation “fly” made the bird fly.

Bird with a collision’s box/sphere - can change the dimensions of the collision box/sphere using the panel on the right.

Figure 43: Viewport View of Bird Pawn Actor



March 24:

Added in a camera component using C++ code to be able to see the bird once we posses it, as without the camera you take on the view from the pawn actor (aka like a first person view, this isn't good here as we want to see the bird and its animations) This camera can be heavily customized including things like hoe far away the camera is, its angle, etc.

I also added in the C++ code and changed/set the key bindings for the forward and backward movement, (w and s). This was done by setting my custom created function "MoveForward" to either 1 (forward) or -1 (backward) by pressing w or s. While neither key is pressed "MoveForward" is set to 0, aka there is no movement.

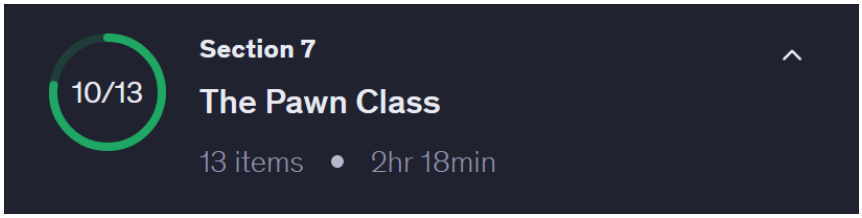
Figure 44: "Bird.cpp" MoveForward Function

```
void ABird::MoveForward(float Value)
{
    if (Controller && (Value != 0.f))
    {
        FVector Forward = GetActorForwardVector();
        AddMovementInput(Forward, Value);
    }
}
```

The animation applied to the bird will cause it to loop through the basic flying animation forever, until we add in other animations that will override the previous animation.

Bird-Flight-Demo.mp4

Udemy has since changed the UI of their website:



END OF PROGRESS REPORT 4

Progress Report 4 Summary:

- Began developing the C++ portion of the Unreal Engine project, including generating C++ project files and resolving early crashes caused by low storage and GPU memory limits.
- Re-enabled World Partition streaming, adjusted cell sizes, regenerated partition data, and significantly improved world-loading behavior.
- Enabled and configured HLOD to reduce draw calls, cut shader load from ~10k to ~3k, and improve distant rendering performance.
- Performed major project cleanup: removed unused Valley of the Ancient projects, deleted old UE installations, moved Steam games, and freed ~415 GB of disk space to stabilize UE performance.
- Adopted a safer C++ compile workflow (closing UE before building, avoiding Live Coding, creating classes in Visual Studio) to prevent blueprint corruption and improve reliability.
- Completed Section 5 and multiple videos in Section 6, learning how to create and customize debugging tools using both UE's built-in systems and C++ code.
- Implemented C++-driven debug object movement, vector manipulation, and rotation, replacing earlier Blueprint-based logic.
- Learned how to define custom controls and expose C++ variables and functions to Blueprints using UPROPERTY and UFUNCTION, enabling editor-side tuning of gameplay values.
- Created a custom pawn actor for a bird asset, added skeletal mesh and animations, configured collision, and implemented forward/backward movement through C++ key bindings.
- Added a fully customizable camera component via C++ to support third-person viewing of the animated bird pawn.
- Reached near the end of Section 7, completing 77 total instructional videos so far and transitioning fully from environment building into gameplay programming.